WEEK4 - TASKS

Cognizant-upGrad Training Program uGE_DotNet FSD(python) Weekly Submission Document

Name: Golla Amulya

Batch Name:B1_Python

Course:uGE DotNet FSD (python)
Training Duration: 16th Feb 2026 - 26th May 2026
Faculty: Kunal

03/09/2026

Project Question-1:
Library Management Console Application

Cognizant is building a **console-based library system** for a training center. Employees can **borrow, return, and search books**.

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Threading.Tasks;

namespace LibraryManagement
{
    // Custom Exception
    class BookNotFoundException : Exception
    {
        public BookNotFoundException(string message) : base(message) { }
    }

    // Book Class
    class Book
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public string Author { get; set; }
        public bool IsAvailable { get; set; } = true;
```

```
    public Queue<string> WaitingUsers = new Queue<string>();
}

class Program
{
    static List<Book> books = new List<Book>();
    static Dictionary<int, Book> bookDictionary = new Dictionary<int, Book>();
    static Stack<string> actions = new Stack<string>();

    static string filePath = "books.txt";

    static async Task Main(string[] args)
    {
        LoadBooks();

        while (true)
        {
            Console.WriteLine("\n1 Add Book");
            Console.WriteLine("2 Borrow Book");
            Console.WriteLine("3 Return Book");
            Console.WriteLine("4 Search Book by Author");
            Console.WriteLine("5 View All Books");
            Console.WriteLine("6 Undo Last Action");
            Console.WriteLine("7 Exit");

            Console.Write("Choose: ");
            int choice = int.Parse(Console.ReadLine());

            try
            {
                switch (choice)
                {
                    case 1:
                        await AddBook();
                        break;

                    case 2:
                        BorrowBook();
                        break;

                    case 3:
                        ReturnBook();
                        break;
```

```
            case 4:
                SearchBook();
                break;

            case 5:
                ViewBooks();
                break;

            case 6:
                UndoAction();
                break;

            case 7:
                return;

            default:
                Console.WriteLine("Invalid option");
                break;
        }
    }
    catch (BookNotFoundException ex)
    {
        Console.WriteLine(ex.Message);
    }
    catch (Exception)
    {
        Console.WriteLine("Invalid Input");
    }
    finally
    {
        Console.WriteLine("Operation Completed\n");
    }
  }
}

// Add Book
static async Task AddBook()
{
    Console.Write("Book Id: ");
    int id = int.Parse(Console.ReadLine());

    Console.Write("Title: ");
    string title = Console.ReadLine();
```

```
    Console.Write("Author: ");
    string author = Console.ReadLine();

    Book book = new Book
    {
        Id = id,
        Title = title,
        Author = author
    };

    books.Add(book);
    bookDictionary[id] = book;

    actions.Push($"ADD:{id}");

    await SaveBooksAsync();

    Console.WriteLine("Book Added");
}

// Borrow Book
static void BorrowBook()
{
    Console.Write("Enter Book Id: ");
    int id = int.Parse(Console.ReadLine());

    if (!bookDictionary.ContainsKey(id))
        throw new BookNotFoundException("Book not found");

    Book book = bookDictionary[id];

    Console.Write("Enter User Name: ");
    string user = Console.ReadLine();

    if (book.IsAvailable)
    {
        book.IsAvailable = false;
        actions.Push($"BORROW:{id}");
        Console.WriteLine($"{user} borrowed {book.Title}");
    }
    else
    {
        book.WaitingUsers.Enqueue(user);
        Console.WriteLine($"{user} added to waiting queue");
```

```
        }
    }

    // Return Book
    static void ReturnBook()
    {
        Console.Write("Enter Book Id: ");
        int id = int.Parse(Console.ReadLine());

        if (!bookDictionary.ContainsKey(id))
            throw new BookNotFoundException("Book not found");

        Book book = bookDictionary[id];

        if (book.WaitingUsers.Count > 0)
        {
            string nextUser = book.WaitingUsers.Dequeue();
            Console.WriteLine($"{nextUser} gets the book now");
        }
        else
        {
            book.IsAvailable = true;
            Console.WriteLine("Book returned and available");
        }
    }

    // LINQ Search
    static void SearchBook()
    {
        Console.Write("Enter Author Name: ");
        string author = Console.ReadLine();

        var result = books.Where(b => b.Author.Equals(author,
StringComparison.OrdinalIgnoreCase));

        foreach (var book in result)
        {
            Console.WriteLine($"{book.Id} {book.Title} {book.Author}
Available:{book.IsAvailable}");
        }
    }

    // View All Books
    static void ViewBooks()
```

```
        {
            foreach (var book in books)
            {
                Console.WriteLine($"{book.Id} {book.Title} {book.Author}
Available:{book.IsAvailable}");
            }
        }

        // Undo Last Action
        static void UndoAction()
        {
            if (actions.Count == 0)
            {
                Console.WriteLine("Nothing to undo");
                return;
            }

            string action = actions.Pop();
            string[] parts = action.Split(':');

            string type = parts[0];
            int id = int.Parse(parts[1]);

            if (type == "ADD")
            {
                Book book = bookDictionary[id];
                books.Remove(book);
                bookDictionary.Remove(id);
                Console.WriteLine("Undo Add Book");
            }

            if (type == "BORROW")
            {
                Book book = bookDictionary[id];
                book.IsAvailable = true;
                Console.WriteLine("Undo Borrow");
            }
        }

        // Save Books Async
        static async Task SaveBooksAsync()
        {
            await Task.Delay(500); // simulate database save
```

```
            using (StreamWriter writer = new StreamWriter(filePath))
            {
                foreach (var book in books)
                {
writer.WriteLine($"{book.Id},{book.Title},{book.Author},{book.IsAvailable}");
                }
            }
        }

        // Load Books
        static void LoadBooks()
        {
            if (!File.Exists(filePath))
                return;

            string[] lines = File.ReadAllLines(filePath);

            foreach (var line in lines)
            {
                string[] data = line.Split(',');

                Book book = new Book
                {
                    Id = int.Parse(data[0]),
                    Title = data[1],
                    Author = data[2],
                    IsAvailable = bool.Parse(data[3])
                };

                books.Add(book);
                bookDictionary[book.Id] = book;
            }
        }
    }
}
```

Project Question-2:

Student Task Tracker

Cognizant wants a **student task tracking system** used in training programs.Each student has tasks assigned and the system tracks **pending, completed, and overdue tasks**.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Text.Json;
using System.Threading.Tasks;

class InvalidTaskException : Exception
{
    public InvalidTaskException(string msg) : base(msg) { }
}

class TaskItem
{
    public int TaskId { get; set; }
    public string Title { get; set; }
    public string StudentName { get; set; }
    public DateTime Deadline { get; set; }
    public bool IsCompleted { get; set; }
}

class Program
{
    static List<TaskItem> tasks = new List<TaskItem>();
    static Dictionary<int, TaskItem> taskMap = new Dictionary<int, TaskItem>();
    static HashSet<string> students = new HashSet<string>();
    static Queue<TaskItem> processQueue = new Queue<TaskItem>();

    static string file = "tasks.json";

    static void Main()
    {
        LoadTasks();

        while (true)
        {
            Console.WriteLine("\n1 Add Task");
            Console.WriteLine("2 Complete Task");
            Console.WriteLine("3 View Tasks");
            Console.WriteLine("4 Overdue Tasks");
            Console.WriteLine("5 Exit");
```

```
        Console.Write("Choice: ");

        int ch = int.Parse(Console.ReadLine());

        try
        {
            switch (ch)
            {
                case 1:
                    AddTask();
                    break;

                case 2:
                    Console.Write("Task Id: ");
                    int id = int.Parse(Console.ReadLine());

                    Task.Run(() => CompleteTask(id));
                    break;

                case 3:
                    ShowTasks();
                    break;

                case 4:
                    ShowOverdue();
                    break;

                case 5:
                    SaveTasks();
                    return;
            }
        }
        catch (InvalidTaskException ex)
        {
            Console.WriteLine(ex.Message);
        }
        catch
        {
            Console.WriteLine("Invalid input");
        }
    }
}

static void AddTask()
```

```
{
    TaskItem t = new TaskItem();

    Console.Write("Task Id: ");
    t.TaskId = int.Parse(Console.ReadLine());

    Console.Write("Title: ");
    t.Title = Console.ReadLine();

    Console.Write("Student: ");
    t.StudentName = Console.ReadLine();

    Console.Write("Deadline (yyyy-mm-dd): ");
    t.Deadline = DateTime.Parse(Console.ReadLine());

    if (t.Deadline < DateTime.Now)
        throw new InvalidTaskException("Deadline already passed");

    tasks.Add(t);
    taskMap[t.TaskId] = t;

    students.Add(t.StudentName);

    processQueue.Enqueue(t);

    Console.WriteLine("Task added");
}

static void CompleteTask(int id)
{
    if (!taskMap.ContainsKey(id))
        throw new InvalidTaskException("Task not found");

    TaskItem t = taskMap[id];

    t.IsCompleted = true;

    Console.WriteLine($"Task {t.TaskId} completed by {t.StudentName}");
}

static void ShowTasks()
{
    foreach (var t in tasks)
    {
```

```
            Console.WriteLine($"{t.TaskId} | {t.Title} | {t.StudentName} |
{t.Deadline.ToShortDateString()} | Completed:{t.IsCompleted}");
        }

        Console.WriteLine("Unique Students: " + students.Count);

        int completed = tasks.Count(x => x.IsCompleted);

        Console.WriteLine("Completed Tasks: " + completed);
    }

    static void ShowOverdue()
    {
        var list = tasks.Where(t => t.Deadline < DateTime.Now && !t.IsCompleted);

        foreach (var t in list)
        {
            Console.WriteLine($"Overdue: {t.Title} ({t.StudentName})");
        }
    }

    static void SaveTasks()
    {
        string json = JsonSerializer.Serialize(tasks);

        File.WriteAllText(file, json);
    }

    static void LoadTasks()
    {
        if (!File.Exists(file))
            return;

        string json = File.ReadAllText(file);

        tasks = JsonSerializer.Deserialize<List<TaskItem>>(json);

        foreach (var t in tasks)
        {
            taskMap[t.TaskId] = t;
            students.Add(t.StudentName);
        }
    }
}
```

03/10/2026

Project Question-3:

Smart Warehouse Inventory & Order Processing System

A logistics company wants a **C# console application** to manage a warehouse where products are stored, orders are processed, and inventory is updated.

The system should support **inventory management, order processing, and reporting** while following **object-oriented design and modern C# practices**.

The system must be designed so that it is **maintainable, scalable, and safe from runtime failures**.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Threading.Tasks;
using System.Data.SqlClient;
//using System.Data.SqlClient;

class OutOfStockException : Exception
{
    public OutOfStockException(string msg) : base(msg) { }
}

class Product
{
    public int ProductId { get; set; }
    public string ProductName { get; set; }
    public string? Category { get; set; }
    public double Price { get; set; }

    private int quantity;

    public int QuantityAvailable
    {
        get { return quantity; }
        set
        {
            if (value < 0)
```

```
            throw new Exception("Quantity cannot be negative");

        quantity = value;
    }
}

public Product(int id, string name, string? category, double price, int qty)
{
    ProductId = id;
    ProductName = name;
    Category = category;
    Price = price;
    QuantityAvailable = qty;
}

public void ReduceStock(int qty)
{
    if (qty > QuantityAvailable)
        throw new OutOfStockException("Not enough stock");

    QuantityAvailable -= qty;
}
}

class Order
{
    public int OrderId { get; set; }
    public string CustomerName { get; set; }
    public List<(int ProductId, int Quantity)> Items = new List<(int, int)>();
    public DateTime OrderDate { get; set; } = DateTime.Now;

    public double CalculateTotal(Dictionary<int, Product> inventory)
    {
        double total = 0;

        foreach (var item in Items)
        {
            Product p = inventory[item.ProductId];
            total += p.Price * item.Quantity;
        }

        return total;
    }
}
```

```
class InventoryRepository
{
    public Dictionary<int, Product> products = new Dictionary<int, Product>();

    string file = "inventory.txt";

    public void AddProduct(Product p)
    {
        products[p.ProductId] = p;
    }

    public void RemoveProduct(int id)
    {
        products.Remove(id);
    }

    public void UpdateStock(int id, int qty)
    {
        if (products.ContainsKey(id))
            products[id].QuantityAvailable = qty;
    }

    public void Save()
    {
        using (StreamWriter sw = new StreamWriter(file))
        {
            foreach (var p in products.Values)
            {

sw.WriteLine($"{p.ProductId},{p.ProductName},{p.Category},{p.Price},{p.Quantity
Available}");
            }
        }
    }

    public void Load()
    {
        if (!File.Exists(file))
            return;

        var lines = File.ReadAllLines(file);

        foreach (var l in lines)
```

```
        {
            var d = l.Split(',');

            Product p = new Product(
                int.Parse(d[0]),
                d[1],
                d[2],
                double.Parse(d[3]),
                int.Parse(d[4])
            );

            products[p.ProductId] = p;
        }
    }
}

class OrderService
{
    public Queue<Order> orderQueue = new Queue<Order>();
    public Stack<Order> shippedOrders = new Stack<Order>();

    InventoryRepository repo;

    string connection =
        "Server=localhost;Database=WarehouseDB;Trusted_Connection=True;";

    public OrderService(InventoryRepository r)
    {
        repo = r;
    }

    public void PlaceOrder(Order o)
    {
        orderQueue.Enqueue(o);
    }

    public async Task ProcessOrders()
    {
        while (orderQueue.Count > 0)
        {
            Order o = orderQueue.Dequeue();

            await Task.Run(() => ProcessSingleOrder(o));
        }
```

```
    }

    void ProcessSingleOrder(Order o)
    {
        foreach (var item in o.Items)
        {
            if (!repo.products.ContainsKey(item.ProductId))
                throw new Exception("Product not found");

            repo.products[item.ProductId].ReduceStock(item.Quantity);
        }

        shippedOrders.Push(o);

        SaveOrderToDb(o);

        Console.WriteLine($"Order {o.OrderId} processed");
    }

    void SaveOrderToDb(Order o)
    {
        try
        {
            using (SqlConnection con = new SqlConnection(connection))
            {
                con.Open();

                string sql =
                    "INSERT INTO Orders(OrderId,CustomerName,OrderDate)
VALUES(@id,@name,@date)";

                SqlCommand cmd = new SqlCommand(sql, con);

                cmd.Parameters.AddWithValue("@id", o.OrderId);
                cmd.Parameters.AddWithValue("@name", o.CustomerName);
                cmd.Parameters.AddWithValue("@date", o.OrderDate);

                cmd.ExecuteNonQuery();
            }
        }
        catch
        {
            Console.WriteLine("Database error");
        }
```

```
    }
}

class Program
{
    static InventoryRepository repo = new InventoryRepository();
    static OrderService orderService = new OrderService(repo);

    static async Task Main()
    {
        repo.Load();

        while (true)
        {
            Console.WriteLine("\n1 Add Product");
            Console.WriteLine("2 View Inventory");
            Console.WriteLine("3 Place Order");
            Console.WriteLine("4 Process Orders");
            Console.WriteLine("5 Low Stock Products");
            Console.WriteLine("6 Save Inventory");
            Console.WriteLine("7 Exit");

            int ch = int.Parse(Console.ReadLine());

            try
            {
                switch (ch)
                {
                    case 1:
                        AddProduct();
                        break;

                    case 2:
                        ViewInventory();
                        break;

                    case 3:
                        PlaceOrder();
                        break;

                    case 4:
                        await orderService.ProcessOrders();
                        break;
```

```
            case 5:
                ShowLowStock();
                break;

            case 6:
                repo.Save();
                break;

            case 7:
                return;
            }
        }
        catch (OutOfStockException e)
        {
            Console.WriteLine(e.Message);
        }
        catch (Exception e)
        {
            Console.WriteLine("Error: " + e.Message);
        }
        finally
        {
            Console.WriteLine("Operation finished");
        }
    }
}

static void AddProduct()
{
    Console.Write("Id: ");
    int id = int.Parse(Console.ReadLine());

    Console.Write("Name: ");
    string name = Console.ReadLine();

    Console.Write("Category: ");
    string? cat = Console.ReadLine();

    Console.Write("Price: ");
    double price = double.Parse(Console.ReadLine());

    Console.Write("Quantity: ");
    int qty = int.Parse(Console.ReadLine());
```

```
        Product p = new Product(id, name, cat, price, qty);

        repo.AddProduct(p);
    }

    static void ViewInventory()
    {
        foreach (var p in repo.products.Values)
        {
            Console.WriteLine($"{p.ProductId} {p.ProductName} {p.Price}
Qty:{p.QuantityAvailable}");
        }
    }

    static void PlaceOrder()
    {
        Order o = new Order();

        Console.Write("Order Id: ");
        o.OrderId = int.Parse(Console.ReadLine());

        Console.Write("Customer: ");
        o.CustomerName = Console.ReadLine();

        while (true)
        {
            Console.Write("Product Id (0 to stop): ");
            int id = int.Parse(Console.ReadLine());

            if (id == 0)
                break;

            Console.Write("Quantity: ");
            int q = int.Parse(Console.ReadLine());

            o.Items.Add((id, q));
        }

        orderService.PlaceOrder(o);
    }

    static void ShowLowStock()
    {
        var list = repo.products.Values.Where(p => p.QuantityAvailable < 10);
```

```
        foreach (var p in list)
        {
            Console.WriteLine($"{p.ProductName} low stock");
        }
    }
}
```

03/11/2026 **JS PRACTICE**

Question-1:Parallel API Data Processor (Async + Array Methods)

Create a program that simulates **fetching data from multiple APIs simultaneously** and processing the results.

```
function getUsers() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { id: 1, name: "Jhansi" },
        { id: 2, name: "Anita" }
      ]);
    }, 1000);
  });
}

function getOrders() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { userId: 1, amount: 500 },
        { userId: 1, amount: 800 },
        { userId: 2, amount: 300 }
      ]);
    }, 1500);
  });
}

function getPayments() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([1, 2, 3]);
    }, 1200);
```

```
  });
}

// main logic
async function runProgram() {

  const [users, orders, payments] = await Promise.all([
    getUsers(),
    getOrders(),
    getPayments()
  ]);

  const result = users.map(u => {

    const total = orders
      .filter(o => o.userId === u.id)
      .reduce((sum, o) => sum + o.amount, 0);

    return { name: u.name, total };
  });

  result.forEach(r => {
    console.log(r.name + " -> " + r.total);
  });

}

runProgram();
```

Question-2:Smart Array Grouping System

Groups transactions by **category,** Calculates **total spending per category,** Sorts the categories **from highest spending to lowest.**

```
const transactions = [
  { category: "Food", amount: 200 },
  { category: "Travel", amount: 500 },
  { category: "Food", amount: 300 },
  { category: "Shopping", amount: 800 },
  { category: "Travel", amount: 200 }
];
```

```
// group & sum amounts
let totals = transactions.reduce((obj, item) => {
  obj[item.category] = (obj[item.category] || 0) + item.amount;
  return obj;
}, {});

// converting object to array, sorting
let sorted = Object.entries(totals).sort((a, b) => b[1] - a[1]);



sorted.forEach(x => {
  console.log(x[0] + " -> " + x[1]);
});
```

Question-3: Async Task Queue (Simulating Multithreading)

Runs **only 3 tasks at a time,** When a task finishes, the next task starts automatically, Continue until all tasks are finished

```
const tasks = Array.from({ length: 10 }, (_, i) => i + 1);
let running = 0;
const limit = 3;

function runTask(id) {
  return new Promise(resolve => {
    console.log("Starting Task " + id);

    const time = Math.floor(Math.random() * 2000) + 1000;

    setTimeout(() => {
      console.log("Completed Task " + id);
      resolve();
    }, time);
  });
}

async function processQueue() {
```

```
  const queue = [...tasks];

  while (queue.length > 0 || running > 0) {

    while (running < limit && queue.length > 0) {
      const id = queue.shift();
      running++;

      runTask(id).then(() => {
        running--;
      });
    }

    await new Promise(r => setTimeout(r, 100));
  }
}

processQueue();
```

Question-4: Dynamic Table Filter (DOM + Array Methods)

Display employees in a **table,** Add a **dropdown filter for department**, When user selects a department, show **only matching employees**, Display **average salary of visible employees**

```
<!DOCTYPE html>
<html>
<head>
<title>Employee Table</title>
</head>
<body>

<h3>Department Filter</h3>

<select id="dept">
  <option value="All">All</option>
  <option value="IT">IT</option>
  <option value="HR">HR</option>
  <option value="Finance">Finance</option>
</select>

<br><br>
```

```
<table border="1" id="empTable">
  <thead>
    <tr>
      <th>Name</th>
      <th>Department</th>
      <th>Salary</th>
    </tr>
  </thead>
  <tbody></tbody>
</table>

<h3 id="avg"></h3>

<script>
const employees = [
 {name:"Jhansi", department:"IT", salary:100000},
 {name:"Manju", department:"HR", salary:50000},
 {name:"Thulasi", department:"IT", salary:120000},
 {name:"Ramya", department:"Finance", salary:65000},
 {name:"Ushali", department:"HR", salary:30000},
 {name:"Naga", department:"Finance", salary:50000},
 {name:"Deekshi", department:"HR", salary:25000},
 {name:"Deva", department:"Finance", salary:70000}
];

const tableBody = document.querySelector("#empTable tbody");
const dropdown = document.getElementById("dept");
const avgText = document.getElementById("avg");

function showEmployees(list){

tableBody.innerHTML = "";

  list.forEach(e => {
    let row = `<tr>
      <td>${e.name}</td>
      <td>${e.department}</td>
      <td>${e.salary}</td>
    </tr>`;
    tableBody.innerHTML += row;
  });

  const avg = list
      .map(x => x.salary)
```

```
    .reduce((a,b) => a+b,0) / list.length;

  avgText.textContent = "Average Salary: " + Math.round(avg);
}

dropdown.addEventListener("change", function(){

  const d = this.value;

  let filtered = employees.filter(e => {
    if(d === "All") return true;
    return e.department === d;
  });

  showEmployees(filtered);

});

showEmployees(employees);
</script>

</body>
</html>
```

Question-5: Real-Time Search Suggestion (Web + Array Processing)

Create a **search suggestion system** similar to Google autocomplete.

```
<!DOCTYPE html>
<html>
<head>
<title>Search Suggestion</title>
<style>
#list div{
  padding:5px;
}
b{
  color:red;
}
```

```
</style>
</head>
<body>

<h3>Product Search</h3>

<input type="text" id="searchBox" placeholder="type product name">

<div id="list"></div>

<script>

const products = [
"laptop",
"lamp",
"laser printer",
"keyboard",
"mouse",
"monitor",
"mobile"
];

const box = document.getElementById("searchBox");
const list = document.getElementById("list");

box.addEventListener("input", function(){

  const text = box.value.toLowerCase();
  list.innerHTML = "";

  if(text === "") return;
```

```
  const result = products.filter(p =>
    p.toLowerCase().startsWith(text)
  );

  result.forEach(p => {

    const match = p.substring(0,text.length);
    const rest = p.substring(text.length);

    const item = document.createElement("div");
    item.innerHTML = "<b>" + match + "</b>" + rest;

    list.appendChild(item);
  });

});
</script>
</body>
</html>
```

Question-6: Bonus Question (Advanced Array Challenge)

Write a program that:

Flattens the array, Removes **duplicates,** Returns **only even numbers**, Sorts them **descending**

```
const numbers = [
  [1,2,3],
  [4,5],
  [6,7,8,9]
];
let flatArray = numbers.flat();
```

```
let unique = [...new Set(flatArray)];
let even = unique.filter(n => n % 2 === 0);
let result = even.sort((a,b) => b - a);
console.log(result);
```

Question-1:

Persistent Multi-Step Form (LocalStorage + Navigation + Validation)

Problem

Create a **3-step form wizard** that saves progress automatically.

```
<!DOCTYPE html>
<html>
<head>
<title>Multi Step Form</title>
<style>
body{font-family:Arial;margin:30px}
.step{display:none}
.active{display:block}
button{margin-top:10px}
.skill{background:#eee;padding:3px;margin:2px;display:inline-block}
</style>
</head>
<body>
<h3>Profile Form</h3>
<div class="step" id="step1">
<h4>Step 1 - Personal Info</h4>

<input id="name" placeholder="Name"><br><br>
<input id="email" placeholder="Email"><br><br>
<input id="age" type="number" placeholder="Age"><br>

<button onclick="nextStep()">Next</button>
```

```
</div>


<div class="step" id="step2">
<h4>Step 2 - Skills</h4>

<input id="skillInput" list="skills" placeholder="Add skill">
<datalist id="skills">
<option>JavaScript</option>
<option>Python</option>
<option>Java</option>
<option>Go</option>
</datalist>

<button onclick="addSkill()">Add</button>

<div id="skillList"></div>

<br>
<button onclick="prevStep()">Previous</button>
<button onclick="nextStep()">Next</button>
</div>


<div class="step" id="step3">
<h4>Step 3 - Profile Strength</h4>

<progress id="progress" max="100" value="0"></progress>
<p id="percent"></p>

<button onclick="prevStep()">Previous</button>
</div>
```

```
<script>

let step = 1
let skills = []

function showStep(){
 document.querySelectorAll(".step").forEach(s=>s.classList.remove("active"))
 document.getElementById("step"+step).classList.add("active")
 updateProgress()
 saveData()
}

function nextStep(){

 if(step===1){

  const nameVal = document.getElementById("name").value
  const emailVal = document.getElementById("email").value
  const ageVal = document.getElementById("age").value

  if(!nameVal || !emailVal || !ageVal){
    alert("Fill all fields")
    return
  }
}

 if(step===2 && skills.length===0){
  alert("Add at least one skill")
  return
 }
```

```
 step++
 showStep()
}

function prevStep(){
 step--
 showStep()
}

function addSkill(){
 let val = skillInput.value
 if(!val) return

 skills.push(val)

 let div=document.createElement("div")
 div.textContent=val
 div.className="skill"
 skillList.appendChild(div)

 skillInput.value=""
 saveData()
}

function updateProgress(){

 let percent=0

 if(document.getElementById("name").value) percent+=25
if(document.getElementById("email").value) percent+=25
if(document.getElementById("age").value) percent+=25
if(skills.length>0) percent+=25
```

```
 progress.value=percent
 percentText = percent+"%"
 document.getElementById("percent").textContent=percentText
}

function saveData(){
 const data={
 name:name.value,
 email:email.value,
 age:age.value,
 skills:skills,
 step:step
 }

 localStorage.setItem("profileForm",JSON.stringify(data))
}

function loadData(){

 const data=JSON.parse(localStorage.getItem("profileForm"))

 if(!data) return

 name.value=data.name||""
 email.value=data.email||""
 age.value=data.age||""
 skills=data.skills||[]
 step=data.step||1

 skills.forEach(s=>{
  let div=document.createElement("div")
```

```
  div.textContent=s
  div.className="skill"
  skillList.appendChild(div)
 })

}

loadData()
showStep()

</script>

</body>
</html>
```

Question-2 **Smart Responsive Product Grid (CSS Grid + Dynamic Filtering)**
Problem

Create a **product catalog page**.

```
<!DOCTYPE html>
<html>
<head>
<title>Product Grid</title>

<style>

body{
font-family:Arial;
padding:20px;
}

select{
padding:6px;
margin-bottom:20px;
}

.grid{
display:grid;
```

```
grid-template-columns:repeat(4,1fr);
gap:15px;
}

.item{
padding:20px;
border:1px solid #ccc;
text-align:center;
background:#f4f4f4;
transition:opacity 0.4s;
}

.hide{
opacity:0;
display:none;
}

@media(max-width:900px){
.grid{
grid-template-columns:repeat(2,1fr);
}
}

@media(max-width:500px){
.grid{
grid-template-columns:1fr;
}
}

</style>
</head>

<body>

<h2>Products</h2>

<select id="filter">
<option value="all">All</option>
<option value="Electronics">Electronics</option>
<option value="Fashion">Fashion</option>
</select>

<div class="grid" id="products">
```

```
<div class="item Electronics">Laptop<br>Electronics</div>
<div class="item Fashion">Shoes<br>Fashion</div>
<div class="item Fashion">Watch<br>Fashion</div>
<div class="item Electronics">Phone<br>Electronics</div>
<div class="item Electronics">Camera<br>Electronics</div>

</div>

<script>

const dropdown = document.getElementById("filter")
const items = document.querySelectorAll(".item")

dropdown.addEventListener("change",function(){

let category = dropdown.value

items.forEach(function(p){

if(category==="all"){
p.classList.remove("hide")
}
else{
if(p.classList.contains(category)){
p.classList.remove("hide")
}
else{
p.classList.add("hide")
}
}

})

})

</script>

</body>
</html>
```

Question-3:

Location-Based Weather Preference System

(Geolocation + Storage + Events)

Create a webpage that detects **user location** and stores preferences.

```
<!DOCTYPE html>
<html>
<head>
<title>Location Preference</title>

<style>
body{
font-family:Arial;
padding:30px;
}

button{
padding:8px 15px;
margin-bottom:15px;
}

#result{
margin-top:10px;
font-weight:bold;
}
</style>

</head>

<body>

<h2>Weather Preference</h2>

<button id="locBtn">Get My Location</button>

<div id="result"></div>

<br><br>

<label>Select Temperature Unit</label><br>

<input type="radio" name="unit" value="Celsius"> Celsius
<input type="radio" name="unit" value="Fahrenheit"> Fahrenheit

<script>
```

```
// button click event
document.getElementById("locBtn").addEventListener("click",getLocation)

function getLocation(){

if(navigator.geolocation){

navigator.geolocation.getCurrentPosition(showPosition,showError)

}
else{
document.getElementById("result").innerText="Geolocation not supported"
}

}

// show coordinates
function showPosition(pos){

let lat = pos.coords.latitude
let lon = pos.coords.longitude

document.getElementById("result").innerText =
"Latitude: "+lat+"  Longitude: "+lon

}

// handle errors
function showError(err){

let msg=""

if(err.code===1){
msg="Permission denied"
}
else if(err.code===2){
msg="Location unavailable"
}
else if(err.code===3){
msg="Request timeout"
}

document.getElementById("result").innerText=msg
```

```
}

// save unit preference
let radios = document.querySelectorAll("input[name='unit']")

radios.forEach(function(r){

r.addEventListener("change",function(){

sessionStorage.setItem("unit",this.value)

})

})

// restore preference
let saved = sessionStorage.getItem("unit")

if(saved){

document.querySelector("input[value='"+saved+"']").checked=true

}

</script>

</body>
</html>
```